

(please go through the red highlighted portion as well. If you find any error, please reach out to me)

Problems Related to Performance:

1. Suppose there are three classes of instructions A, B, and C in a particular instruction set architecture with CPIs 2, 1.5, and 2.2 respectively. The number of instructions(IC) from each class in two separate programs is as follows:

Programs	A	B	C
P1(IC)	100	110	116
P2(IC)	112	100	40

Consider that both programs run on the same device with a clock frequency of 2 GHz. Both P1 and P2 require 450 ns to be completed.

- (i) Find out the affected and unaffected times for both programs. [2]
- (ii) Find the improvement factor for P1 and P2 if we get the total program completion time improved by 3x. [6]
- (iii) If P2 now has the same execution time as P1 what should be the number of instructions (IC) for instruction class C? [2]

Solution

Affected time is the time spent on instructions that we might improve (e.g., by reducing their CPI). Unaffected time is the time spent on instructions that are not improved

- **Affected Time (CPU Time):** The time taken to execute the instructions, calculated as:

$$\text{CPU Time} = \text{IC} \times \text{CPI} \times \text{Clock Cycle Time}$$

$$\text{Where Clock Cycle Time} = \frac{1}{\text{Clock Frequency}} = \frac{1}{2 \text{ GHz}} = 0.5 \text{ ns.}$$

- **Unaffected Time:** The time not spent executing instructions, calculated as:

$$\text{Unaffected Time} = \text{Total Time} - \text{CPU Time}$$

- A $\rightarrow$ CPI = 2
- B $\rightarrow$ CPI = 1.5
- C $\rightarrow$ CPI = 2.2

Programs:

- P1: A=100, B=110, C=116
- P2: A=112, B=100, C=40

Clock frequency = 2 GHz (Clock cycle = 0.5 ns).

Both programs complete in **450 ns**.

For P1:

Total cycles = $\{100(2) + 110(1.5) + 116(2.2)\} / 2 \times 10^9 = 310.1 \text{ ns}$

For P2:

Total cycles = $\{112(2) + 100(1.5) + 40(2.2)\} / 2 \times 10^9 = 231 \text{ ns}$

Both take 450 ns.

That means actual execution is **longer than base execution** → extra overhead.

- **Affected time** = Base execution time (calculated).
- **Unaffected time** = Extra overhead = (450 – base time).

So:

- P1: unaffected = 310.1 ns, affected = $450 - 310.1 = \mathbf{139.9 \text{ ns}}$
- P2: unaffected = 231 ns, affected = $450 - 231 = \mathbf{219 \text{ ns}}$
- P1: Affected = 139.9 ns, Unaffected = 310.1 ns
- P2: Affected = 219 ns, Unaffected = 231 ns

(ii)

Improvement factor with $3\times$ speedup

If improved execution time = $(450 \div 3) = 150 \text{ ns}$.

$T_{\text{improved}} = 150 \text{ ns}$

For P1,

$$150 = (310.1 / k) + 139.9$$

$$K = 31$$

The value of K is negative. So this speedup is not possible.

For P2,

$$150 = (231 / k) + 219$$

$$k = 231 / (150 - 219) = -3.348$$

The value of K is negative. So the speedup is not possible.

(iii)

If P2 has same execution time as P1

We want: P2 execution time = P1 execution time = 310.1 ns (base).

Convert to cycles:

$$310.1 \div 0.5 = \mathbf{620.2 \text{ cycles.}}$$

We already have for P2 (without change):

$$112(2) + 100(1.5) + (C \times 2.2) = 224 + 150 + 2.2C$$

$$C \approx 112$$

Instruction class C should have **112 instructions**.

. (a) A processor with a clock rate of 3 GHz executes a program with the following instruction distribution:

	FP	INT	L/S	Branch
Instruction Count	150	20	50	35
CPI	5	3	4	2

We want to make the program run three times faster. Find the improvement factor of the given program if we use a better ALU that can only improve the time for FP instructions. [5]

Instruction Count:

FP: Count = 150, CPI = 5

INT: Count = 20, CPI = 3

L/S: Count = 50, CPI = 4

Branch: Count = 35, CPI = 2

Total cycles= $(150 \times 5) + (20 \times 3) + (50 \times 4) + (35 \times 2) = 750 + 60 + 200 + 70 = 1080$ cycles

Execution Time = Total Cycle / Clock Rate = $1080 / (3 \times 10^9) = 360$ ns

$T_{\text{improved}} = 360 / 3 = 120$ ns

If we want to improve the FP instruction $\rightarrow (150 \times 5) / (3 \times 10^9) = 250$ ns

$T_{\text{affected}} = 250$ ns

$T_{\text{unaffected}} = 360 - 250 = 110$ ns

Using Amdahls's law,

$120 = 250 / k + 110$

$K = 250 / (120 - 110) = 25$

- (b) Suppose there are three instruction classes, X, Y, and Z, in a given instruction set architecture with CPIs of 1.5, 2.5, and 3.0 respectively. Two programs, Q1 and Q2, execute on this architecture, with the number of instructions from each class as follows:

Programs	Instruction Class		
	X	Y	Z
Q1	30	40	25
Q2	30	30	70

If both programs are executed on the same device with a clock frequency of 2.7 GHz, determine which program runs faster and by how much. [5]

CPI for classes: $X = 1.5$, $Y = 2.5$, $Z = 3.0$

Clock frequency = 2.7 GHz

Instruction counts:

Q1: $X=30$, $Y=40$, $Z=25$

Q2: $X=30$, $Y=30$, $Z=70$

Total cycles of Q1 = $(30 \times 1.5) + (40 \times 2.5) + (25 \times 3.0) = 45 + 100 + 75 = 220$ cycles

Total cycles of Q2 = $(30 \times 1.5) + (30 \times 2.5) + (70 \times 3.0) = 45 + 75 + 210 = 330$ cycles

Execution time of Q1 = $220 / (2.7 \times 10^9) = 81.4$ ns

Execution time of Q2 = $330 / (2.7 \times 10^9) = 122$ ns

Q1 is faster than Q2.

1. Consider the following MIPS code:

```
1      block:
2          add $sp, $sp, -8
3          sw $ra, $zero($sp)
4          sw $s0, 4($sp)
5
6          addi $v0, 0, 0
7          addi $s0, $zero, 1
8
9      loop:
10         slti $t0, $a0, s0
11         bne $t0, $zero, end_loop
12         add $v0, $v0, $s0
13         addi $s0, $s0, 1
14         jump loop
15
16     end_loop:
17         lw $ra, 4($sp)
18         lw $s0, 8($sp)
19         addi $sp, $sp, 12
20         jr $ra
```

[N.B. You should assume the value of \$a0 = 3]

- (a) **Find** the errors and fix them in the above code. [4]
- (b) As you get error-free code from 1a, **Write** the value of \$v0 and \$s0 registers after executing the program. [2]
- (c) Consider a CPU with a 4GHz clock rate and CPI in the following table-1. **Calculate** the CPU time for executing the program 1a. [4]
- (d) Consider program 1a running on another computer that requires 160ns, with 40ns spent executing FP instructions, 90ns executed L/S instructions, and 30ns spent executing branch instructions. What is the improvement factor using Amdahl's law if we only improve the performance of L/S instructions using a better ALU to get the program completion time improved by 2x? [4]

Instruction	CPI
add	2
addi	3
sub	2
lw	4
sw	4
sll	1
srl	1
slt	4
stli	8
beq	4
bne	4
j	8
jr	4
jal	4

Table 1: CPI for each instruction for program 1a

Corrected Code:

block:

add \$sp, \$sp, -8

sw \$ra, \$zero(\$sp) # Error 1: \$zero is not a valid base register for sw; should be \$sp

sw \$s0, 4(\$sp)

addi \$v0, \$0, 0 # Correct: \$v0 = 0

addi \$s0, \$zero, 1 # Correct: \$s0 = 1

loop:

slti \$t0, \$a0, s0 # Error 2: s0 should be \$s0

bne \$t0, \$zero, end_loop

add \$v0, \$v0, \$s0

addi \$s0, \$s0, 1

j loop # Error 3: "jump" should be "j"

end_loop:

lw \$ra, 4(\$sp) # Error 4: offset 4? But we stored \$ra at 0(\$sp) and \$s0 at 4(\$sp)

lw \$s0, 8(\$sp) # Error 5: offset 8? But stack only allocated 8 bytes, so valid offsets are 0 and 4.

addi \$sp, \$sp, 12 # Error 6: we allocated 8 bytes, so we should deallocate 8, not 12.

jr \$ra

(b)

\$v0 = 6 (the sum)

\$s0 = 4 (i becomes 4 before breaking)

(c)

The Loop runs 3 times. So we need to calculate the total instruction count for each instruction based on how many times they are being executed.

add: CPI=2

addi: CPI=3

sw: CPI=4

lw: CPI=4

slti: CPI=8

bne: CPI=4

j: CPI=8

jr: CPI=4

So total cycles:

add: $4 * 2 = 8$

addi: $6 * 3 = 18$

sw: $2 * 4 = 8$

lw: $2 * 4 = 8$

slti: $3 * 8 = 24$

bne: $3 * 4 = 12$

j: $2 * 8 = 16$ (j loop)

jr: $1 * 4 = 4$

Total cycles = $8+18+8+8+24+12+16+4 = 98$ cycles

Execution Time = $98 / (4 \times 10^9) = 24.5$ ns

(c)

Total time = 160 ns, FP time = 40 ns, L/S time = 90 ns, Branch time = 30 ns

We want to improve L/S instructions to make total time 2x faster, i.e., new total time = 80 ns.

Let improvement factor for L/S be k (so new L/S time = $90/k$).

If we can improve L/S instruction, $T_{affected} = 90$ ns

Unaffected time = Total Response Time - (Time for FP and Branch) = $160 - 40 - 30 = 90$ ns

Using Amdahl's law,

$$80 = 90 / k + 90$$

$$K = 90 / (80 - 90) = -9$$

K is negative, so improvement is not possible.

Problems Related to MIPS:

2. Consider the following C function and assign registers according to your requirements.

```

1 void calculate(int i, int k){
2   int a, b=15;
3   a = ((b%8)|i)&(~k);
4 }

```

- (a) **Convert** the code to the corresponding MIPS assembly instructions. [4]
 (b) **Convert** the first 8 lines of MIPS assembly instructions to the corresponding machine code. No need to convert it into binary. [4]
 (c) Can we write the below code in MIPS? **Justify** with proper explanation. [3]

```

1 sll $t0, $s1, 40

```

- (d) Assume that processor X has 256 registers. The size of MIPS instruction is 32 bits and 6 bits are reserved for opcode. The structure for beq instruction is given in the table-1. **Find** out the maximum value of Y that can be written in the instruction below. [4]

opcode	rs	rt	address
--------	----	----	---------

Table 1: Structure of beq instruction

```

1 beq $s5, $zero, Y

```

(a)

$i \rightarrow \$a0$

$k \rightarrow \$a1$

Local variables:

$a \rightarrow \$s0$

$b \rightarrow \$s1$

Solution:

Calculate:

addi \$sp, \$sp, -8 # make space for locals (a, b)

sw \$t0, 4(\$sp) # b= \$s0

```
sw $t6, 0($sp)    # a = $s1
```

If divisor is a power of 2 (2^n), then

$$x \% 2^n = x \& (2^n - 1)$$

This works because binary division by 2^n just shifts, and the remainder is the lower n bits.

```
addi $s0, $zero, 15 #b=15
andi $t2, $s0, 7 #t0 = b%8
or $t3, $t2, $a0
and $s1 $t4, $t5    # result
lw $t0, 4($sp)      # restore b= $s0
lw $t6, 0($sp)      # restore a = $s1
addi $sp, $sp, 8     # restore stack
jr $ra
```

(c)

Cannot encode `sll $t0, $s1, 40` directly in MIPS.

Reason: `shamt` field is only 5 bits, so maximum shift = 31.

(d)

Processor X has 256 registers $\rightarrow$ needs 8 bits per register (since Instruction size = 32 bits.

Opcode = 6 bits.

Instruction format (like MIPS I-type):

| opcode (6) | rs (8) | rt (8) | address (10) |

(because $32 - 6 - 8 - 8 = 10$ bits left for address).

address field = 10 bits → can represent values from 1023 (signed 2's complement).

But in branch instructions, the offset is in words, not bytes (shifted left by 2 when used).

So:

Maximum immediate value = 1023 (decimal).

Meaning

Equivalent byte offset = $1023 \times 4 = 4092$

There might be some confusion regarding whether we will consider the negative value since in the constant field, the sign bits are considered. In order to avoid the confusion,

if the question says,

it is an I type instruction -> consider sign bit

If it says only Beq, Bne, Lw, Sw -> No need to consider sign bit

If it says andi or addi (any immediate instruction) -> then consider sign bit

2. Consider the following C function and assign registers according to your requirements.

```
1  int arr_sum(int arr[],int n){
2  int sum = 0, a = 10, b = -7, c = 12;
3  for(int i=0;i<n;i++){
4      if(arr[i]<=0){
5          b = (a < 15) && ( b != c)
6      }
7      else{
8          b = ((~15 + c) & 1)
9      }
10     sum += arr[i+b]
11 }
12 return sum;
13 }
```

- (a) **Convert** the code to the corresponding MIPS assembly instructions. [8]
- (b) **Convert** the first 8 lines of MIPS assembly instructions to the corresponding machine code. No need to convert it into binary. [4]
- (c) Assume that the processor has 64 registers. The size of MIPS instruction is 32 bits and 6 bits are reserved for opcode. The structure for lw instruction is given in the table-1. **Find** out the maximum index number for an array in MIPS. [3]

opcode	rs	rt	constant
--------	----	----	----------

Table 1: Structure of lw instruction

(a)

arr_sum:

```

addi $sp, $sp, -24
sw $s0, 0($sp)
sw $s1, 4($sp)
sw $s2, 8($sp)
sw $s3, 12($sp)
sw $s4, 16($sp)
sw $ra, 20($sp)

```

```

# Initialize locals
add $s0, $zero, $zero # sum = 0
addi $s1, $zero, 10   # a = 10
addi $s2, $zero, -7   # b = -7
addi $s3, $zero, 12   # c = 12
add $s4, $zero, $zero # i = 0

```

loop:

```

slt $t0, $s4, $a1      # i < n?
beq $t0, $zero, end_loop

```

```

# Load arr[i] into $t1
sll $t1, $s4, 2        # i * 4
add $t1, $a0, $t1      # &arr[i]
lw $t1, 0($t1)         # $t1 = arr[i]

```

```

# Check if arr[i] <= 0
slt $t2, $zero, $t1    # 0 < arr[i]? (so arr[i] > 0)

```

```

$t1 = arr[i]
0 < arr[i] -> t2 = 1
0 >= arr[i] -> t2 = 0

```

Note that \$t2 will only be 1 if the first value is lesser than the second value. It is not considering whether they are equal or not. So instead of checking $\text{arr}[i] \leq 0$, it is much more easier to check if $0 < \text{arr}[i]$, this also reduces the lines of code you will use.

```

bne $t2, $zero, else

```

```

# Then block: b = (a < 15) && (b != c)
# Evaluate (a < 15)

```

```
addi $t3, $zero, 15    # $t3 = 15
slt $t4, $s1, $t3      # 1 if a < 15, else 0
```

```
# Evaluate (b != c)
```

```
seq $t5, $s2, $s3      # 1 if b == c, else 0
```

```
xori $t5, $t5, 1       # now 1 if b != c, else 0
```

— No need to use this instruction

For (b != c)

```
sub $t8, $s2, $s3
```

```
beq $t8, $zero, BANDCAREEQUAL
```

```
BANDCARENOTEQUAL:
```

```
addi $t5, $zero, 1
```

```
j REST
```

```
BANDCAREEQUAL
```

```
addi $t5, $zero, 0
```

```
REST:
```

```
# Logical AND
```

```
and $s2, $t4, $t5      # b = (a < 15) && (b != c)
```

```
j endif
```

else:

```
# Else block: b = ((~15 + c) & 1)
```

```
addi $t6, $zero, 15    # $t6 = 15
```

```
nor $t6, $t6, $zero     # ~15 (nor with zero gives bitwise not)
```

```
add $t6, $t6, $s3       # ~15 + c
```

```
andi $s2, $t6, 1       # & 1
```

endif:

```
# Now compute address of arr[i + b]
```

```
add $t7, $s4, $s2       # i + b
```

```
sll $t7, $t7, 2         # (i+b)*4
```

```
add $t7, $a0, $t7       # &arr[i+b]
```

```
lw $t8, 0($t7)          # $t8 = arr[i+b]
```

```
add $s0, $s0, $t8       # sum += arr[i+b]
```

```
addi $s4, $s4, 1        # i++
```

```
j loop
```

```
end_loop:
    add $v0, $zero, $s0    # return sum
```

```
lw $s0, 0($sp)
lw $s1, 4($sp)
lw $s2, 8($sp)
lw $s3, 12($sp)
lw $s4, 16($sp)
lw $ra, 20($sp)
addi $sp, $sp, 24
jr $ra
```

(c)

Given 64 registers, so each registers take $\log_2(64) = 6$ bits.
Opcode= 6 bits. Rs, Rt= 6 bits.

Size of constant field = $32 - 6 - 6 - 6 = 14$

Max Field Size = $(2^{14} - 1) = 16383$

Maximum Index would be $16383 / 4 = 4095$

Note that,

When we were calculating the memory label, we had to multiply by 4, because when converting from MIPS to machine code, we always divided the memory label by 4. However, in this question, it was asked to calculate the maximum index of array. In the contrast field, the offset of array was placed.

Now offset = 4 x index number.

So index number = offset / 4

Problems Related to Multiplication and Division Algorithm

3. (a) Multiply two numbers A by B using the optimized multiplication algorithm, where A = 7 and B = 9. [4]
 (b) Using the optimized multiplication algorithm if we multiply two numbers of 5 bits, then what will the size of the product register? [1]

(a)

Step	Multiplicand	Multiplier	LSB of Multiplier	Action	Product
0	7	1001	1	Add	7
1	14	0100	0	Skip	7
2	28	0010	0	Skip	7
3	56	0001	1	Add	63
4	112	0000	0	Done	63

(b)

Maximum 5-bit number = 11111 = 31

Maximum product = $31 \times 31 = 961$

How many bits do we need to store 961 in binary?

$$2^9 = 512 < 961 < 1024 = 2^{10}$$

So we need 10 bits to store the product.

But products are always stored in registers in power of 2. So size of product register is 16.

Shortcut to calculate size of product = size of Multiplicand + size of multiplier

In this case,

Multiplicand bit 5, multiplier bit = 5

Product bits = $5 + 5 = 10$

- (b) Assume we want to multiply three numbers A, B and C. Where A=10, B=11 and C=A*B. What will be minimum size of the product register we need to accomplish this multiplication? [1]

$$\text{Product} = A \times B \times C = A \times B \times (A \times B) = 10 \times 11 \times 10 \times 11 = 12100$$

To store 12100, we need 14 bits. Minimum size of product is 14 bits.
Product register (power of two) = 16 bits.

For division algorithm simulation, please check your mid term slide portion, where a simulation is posted in a PDF file.

All the best to everyone for your mid term preparation.